Target - Crackme 3_64

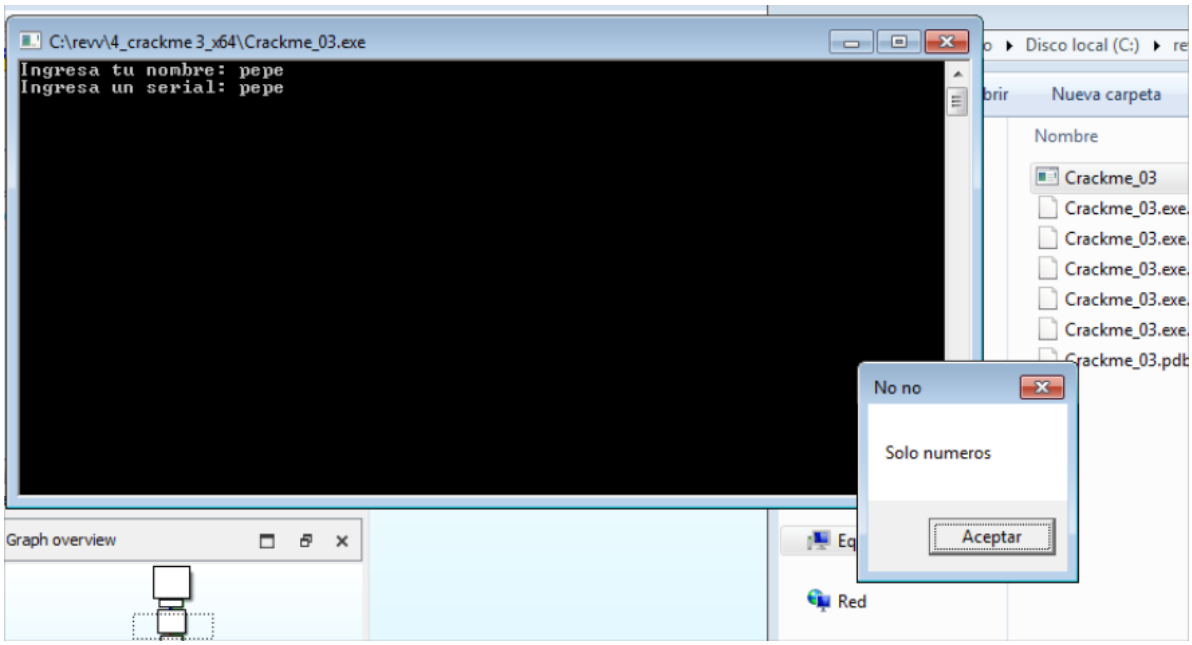
Eugenio D. Deluca (KarasuRØØT) – Date 12/08/2024

This mini-tutorial attempts, in some way, to explain how to solve the crackme

"crackme3_64". What is it like?

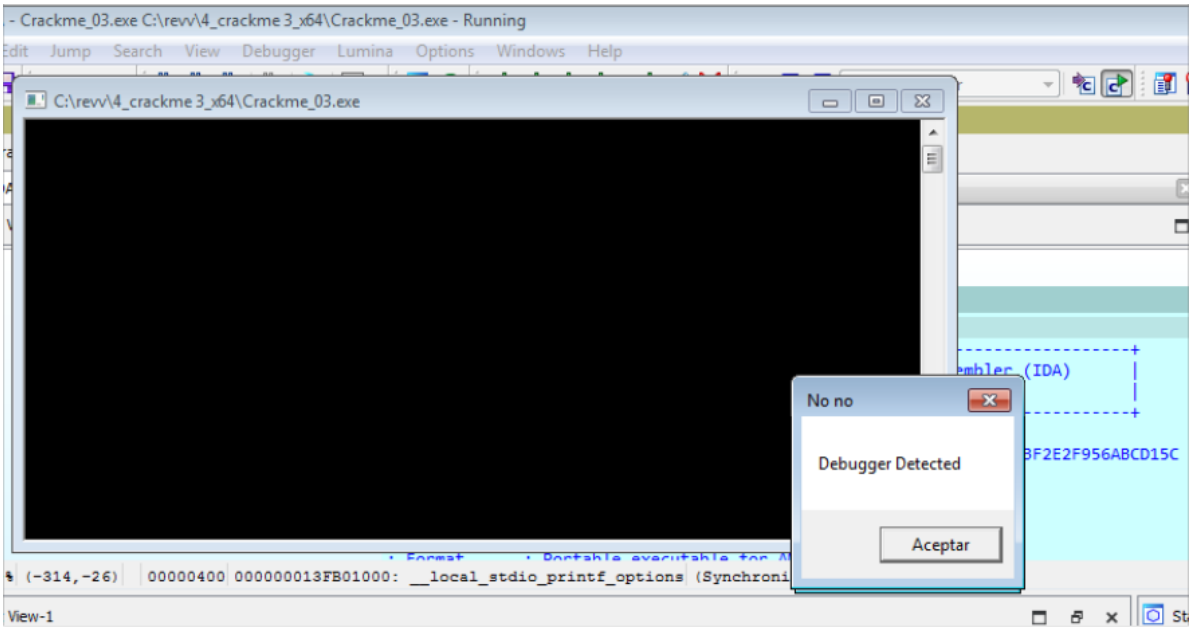
Well, when you start it—either by double-clicking or through the command console—a window opens up asking the user for a name, and then it asks for a serial number.

It verifies that only numbers can be entered as the serial, otherwise a pop-up appears stating "Solo numeros".



I started out using x64dbg—the executable is 64-bit.

If I try to debug it with x64dbg or with the built-in debugger in IDA, I get this message:



Even though that message appears, it lets you keep operating at first.

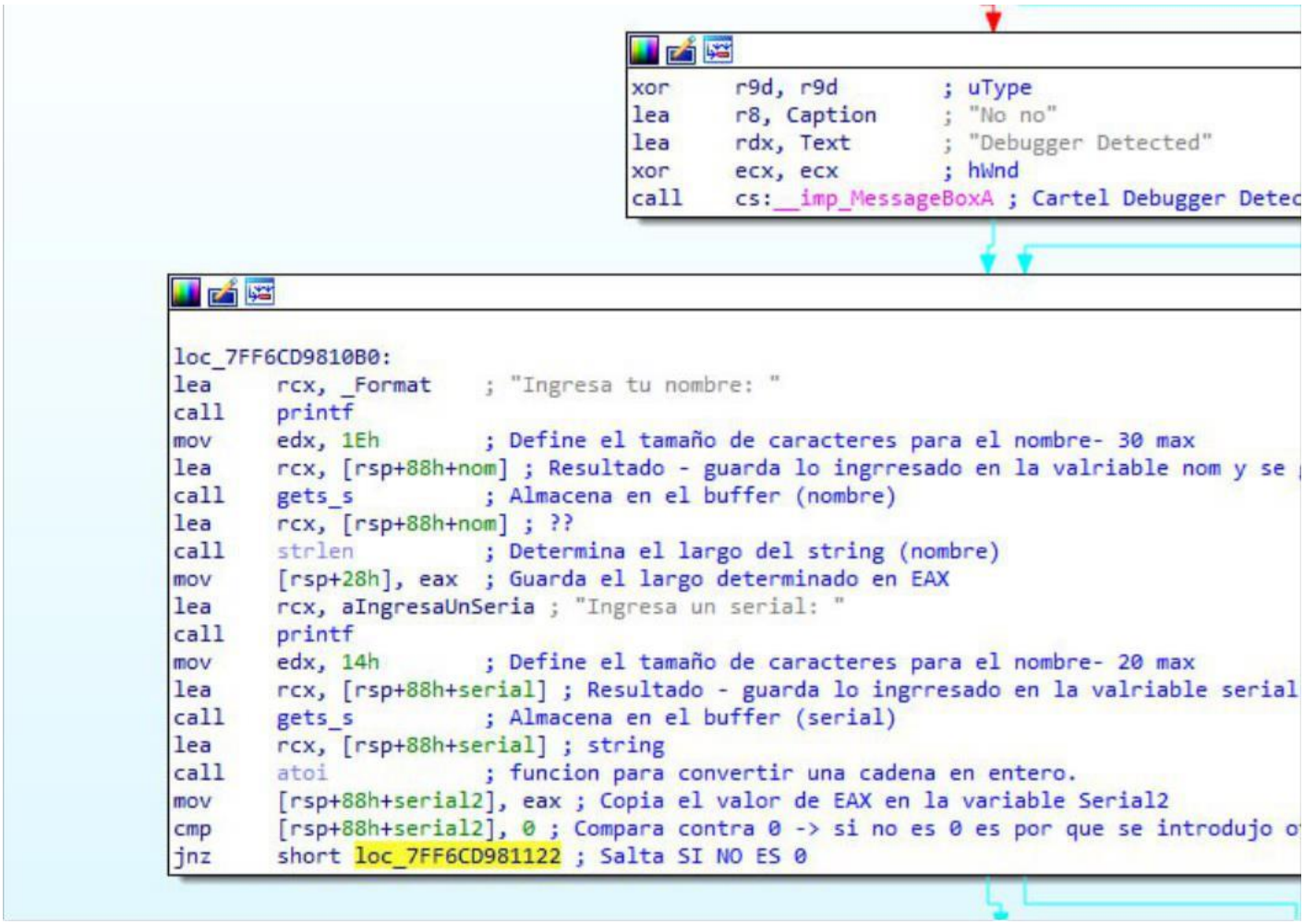
Still, I will start making comments in the IDA diagram to get an idea of what it does

```
; int __fastcall main(int argc, const char **argv, const char **envp)
main proc near

resultado= dword ptr -68h
x= dword ptr -64h
largo= dword ptr -60h
serial2= dword ptr -5Ch
debu= dword ptr -58h
cons= dword ptr -54h
serial= byte ptr -50h
nom= byte ptr -38h
var_18= qword ptr -18h

; __unwind { // __GSHandlerCheck
sub     rsp, 88h
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+88h+var_18], rax
mov     [rsp+88h+resultado], 0 ; Inicializa en 0 la variable resultado
mov     [rsp+88h+cons], 65 ; La constante vale  41h --> es el caracter A en hexa
call    cs:__imp_IsDebuggerPresent ; Llama a la funcion para detectar si hay un debugger o no
mov     [rsp+88h+debu], eax
cmp     [rsp+88h+debu], 1 ; Compara el resultado de debu - variable que es usada en la llamada para ver
jnz     short loc_7FF6CD9810B0 ; Salta SI NO ES 0
```

The important details that I notice at first are two things:

1st -> It has a function call to check for the presence of a debugger.

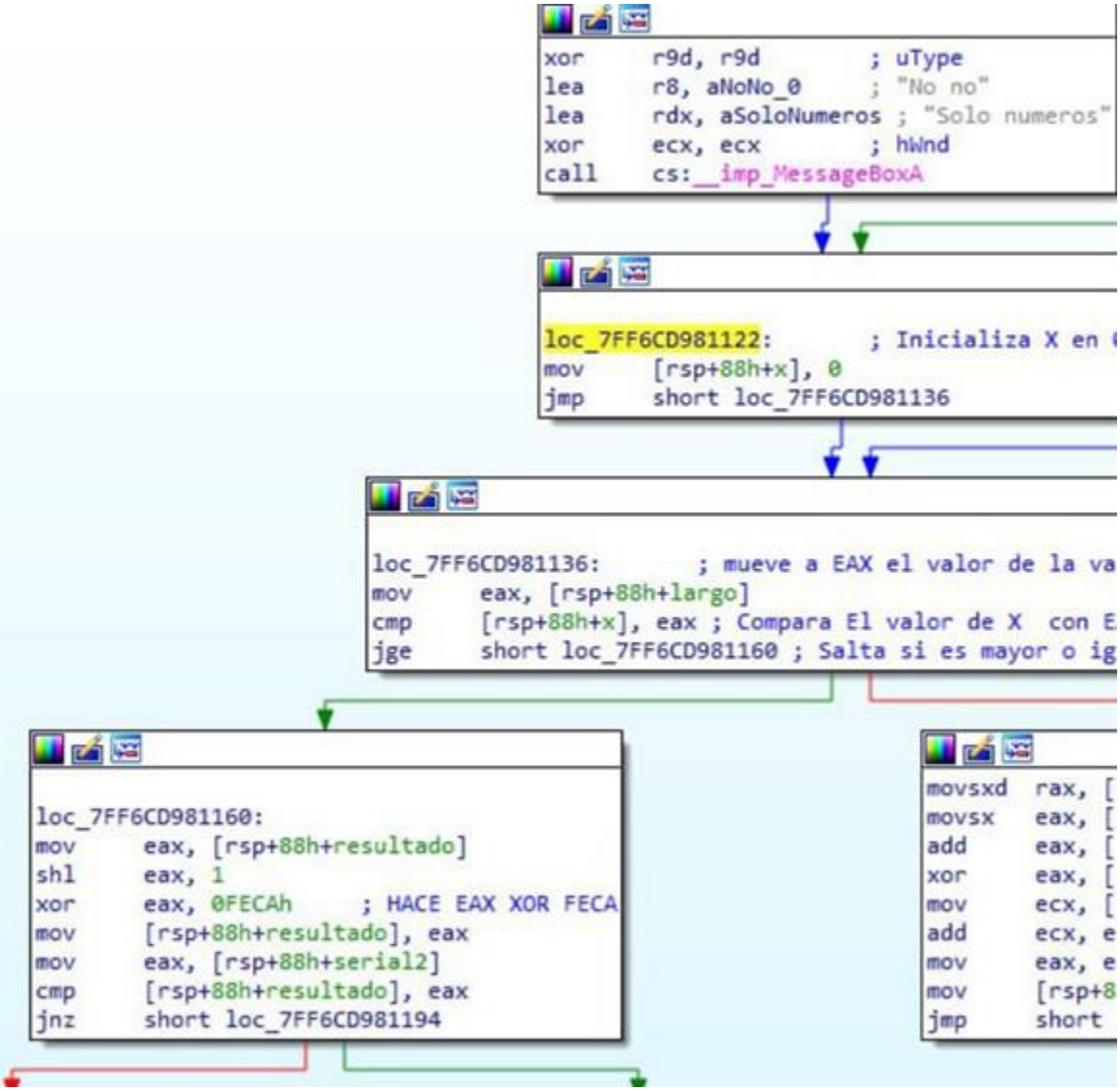
2nd -> The value 41h (the character 'A' in hex) is copied to the variable "cons".



In the next block, it basically takes the input from the keyboard (both the name and the serial) and stores each one in a buffer (defining a size of 30 and 20 respectively)—then it converts them from string to integer using the ATOI function.

Note that the serial we entered is saved in the RCX register—it also compares and detects here whether the serial is empty or not.

It also determines the length of the name (for example, "pepe" has 4 characters).



Next -> It initializes the variable 'x' to 0 and enters -> Then it moves the value corresponding to the length of the name into EAX (following the example with the username "pepe", EAX = 00000004).

It compares the variable 'x' with EAX (4); since it is not greater than or equal to, it enters the loop, and here comes the "trick":

First, it will take the string entered letter by letter (keep in mind that it will handle it as HEX) -> This means that: "pepe" in HEX would be: 70 65 70 65.

Then, what it is going to do is add 4 to each letter (as mentioned before, since that is the length of the string), and it applies an XOR 41 (which is the "cons" variable) to that result.

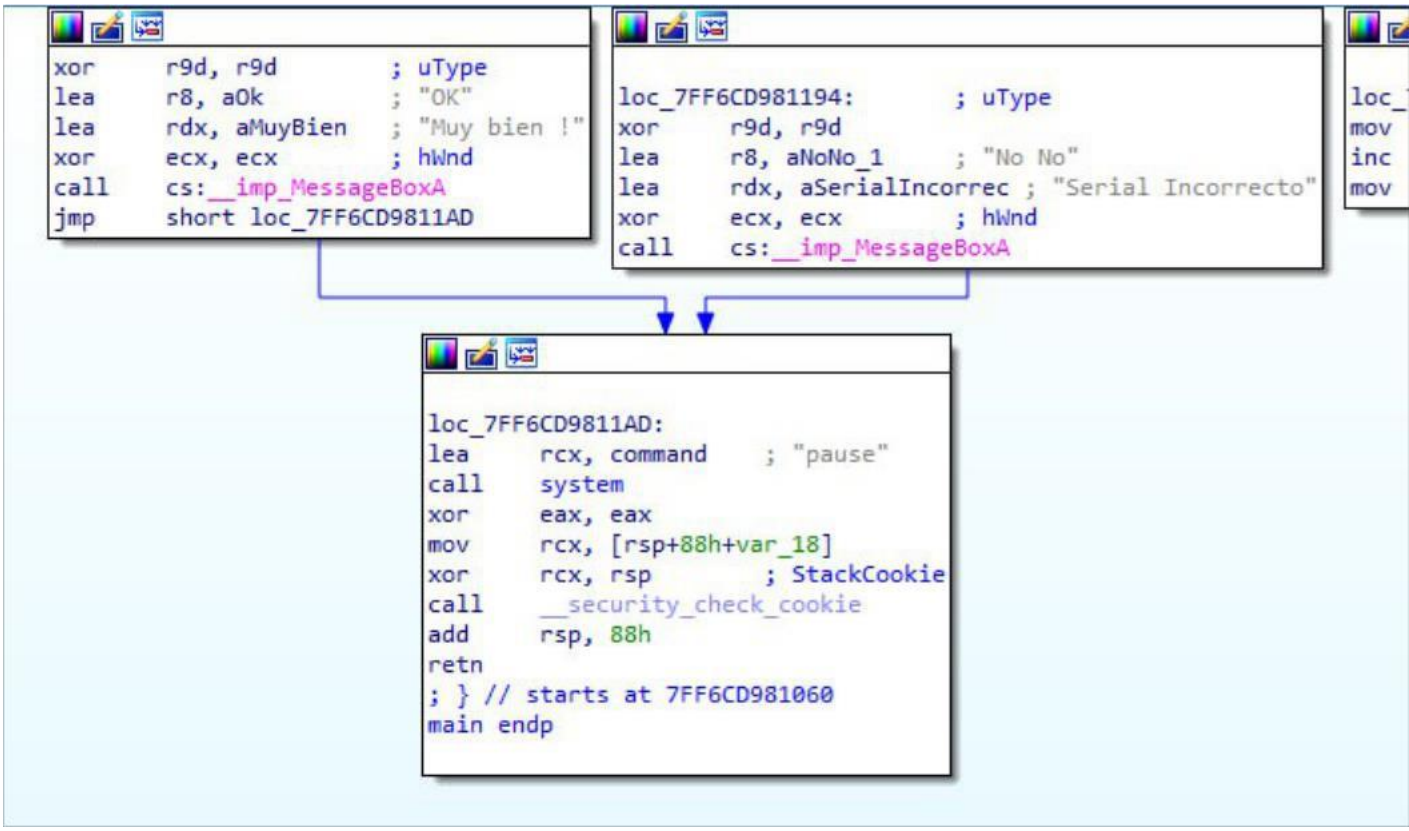
Afterwards, it will save it in the "resultado" variable -> It copies it to ECX.

And then it continues like this until it finishes and goes to the counter, which increments X by +1 -> and goes back to the beginning to compare—this is how it loops through the string and performs the corresponding calculations.

Upon finishing the loop, it jumps to the part that says `loc_7FF6CD981160`—that is, to the left side.

Then, it takes the "resultado" variable and multiplies it by 2 -> this is due to the line `shl eax, 1`. Next, it applies an XOR FECA (which is 65226 in decimal) and saves it.

Finally, it compares the serial we initially entered against the serial it built based on the operations performed on the username. If they match (=), it shows the good-boy message "Muy Bien"—otherwise, it shows the bad-boy message "Serial Incorrecto", as shown in the following image.



That being the case, let's review in a simpler way what this crackme does internally:

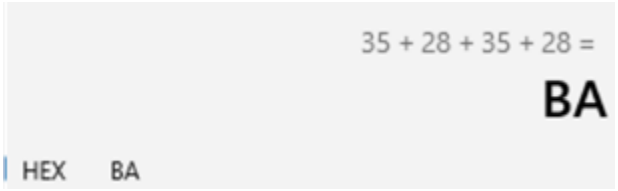
1st: It asks us for a name and a serial number X. Name: pepe

Then, what it does is get the length of the string—as we said, it is 4.

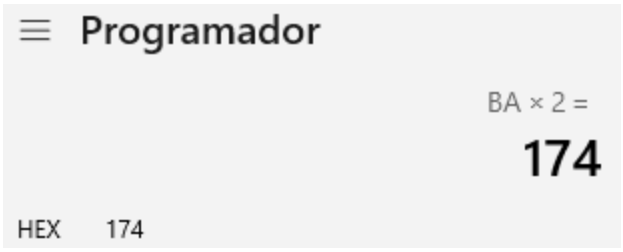
Next, it converts "pepe" into its corresponding hexadecimal values: 70 65 70 65. Then, letter by letter, it performs the operations:

- 70 + 4 = 74 -> 74 XOR 41 = 35
- 65 + 4 = 69 -> 69 XOR 41 = 28
- 70 + 4 = 74 -> 74 XOR 41 = 35
- 65 + 4 = 69 -> 69 XOR 41 = 28

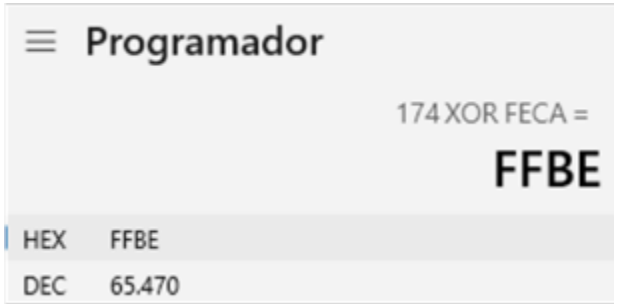
If we sum all the results



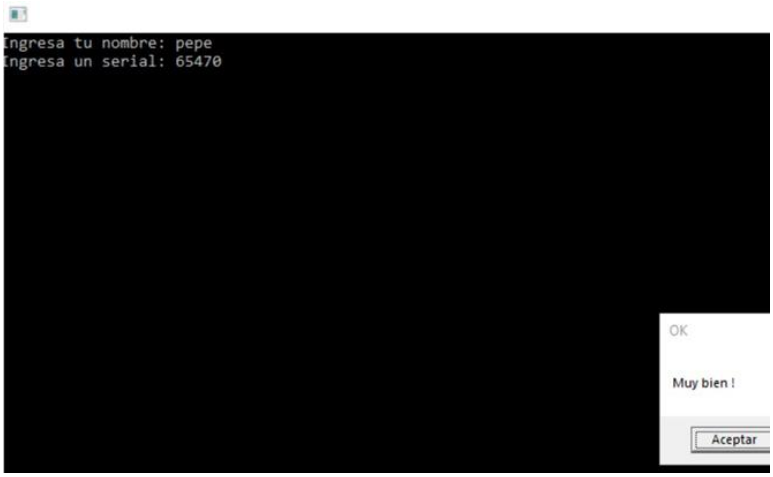
Therefore, BA is what EAX is worth so far in order to continue performing operations on it -> "BA" is multiplied by 2 =



Then, 174 XOR FECA is performed

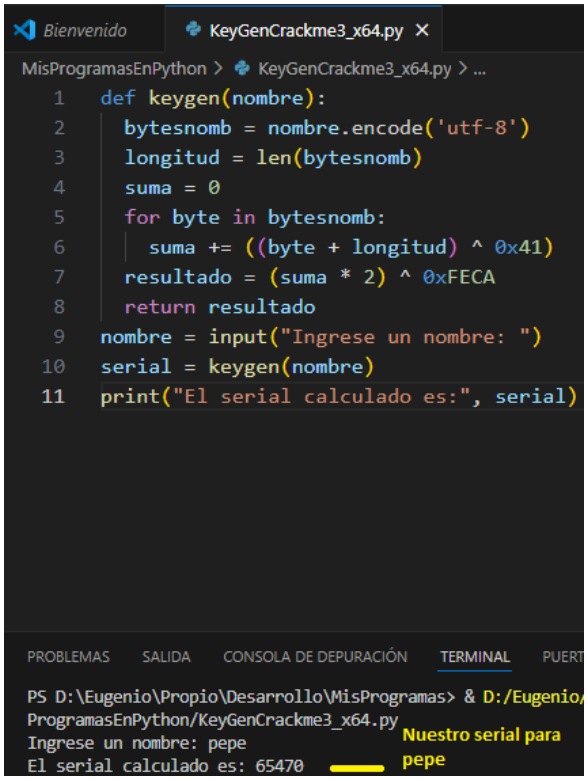


Which means that our serial for the name "pepe" is 65470—Let's verify it:



Great—so with a little help, we wrote the script to be able to use any name and have it give us the corresponding serial number:

The script is written in Python:



It is a fairly simple script that prompts the user for the desired name and performs all the previously mentioned operations, returning the serial number for the specified name.